

Spatio-temporal Neural Structural Causal Models for Bike Flow Prediction

GDL 2025

Group id: STNSCM

Project id: STNSCM

Damiano Ficard, Stefano Quagga, Claudio Ricci
{ficard, quagga, ricci1}@usi.ch

Abstract

This reproducibility study validates the findings of "Spatio-temporal Neural Structural Causal Models for Bike Flow Prediction" [1] by systematically testing its key claims. We first replicate the authors' experiments, confirming their model's superior performance in bike flow prediction, particularly under external fluctuations. To highlight the limitations of traditional spatio-temporal methods, we implement a dummy baseline model and an older graph-based model [2] that separately processes spatial and temporal dependencies. Our experiments show that these approaches, while effective at capturing basic correlations, fail to model complex nonlinear spatio-temporal causality, leading to poorer generalization. Finally, we conduct ablation tests to assess the contributions of the front-door criterion and counterfactual reasoning modules, demonstrating their necessity for robust predictions. Our results reinforce the original paper's conclusions while providing deeper insights into the advantages of causal modeling in bike-sharing systems.

1 Introduction

The paper under review, "Spatio-temporal neural structural causal models for bike flow prediction" [1], proposes a new model called STNSCM (Spatio-temporal Neural Structural Causal Model) for predicting bicycle flow in bike-sharing systems. The high frequency and randomness of bicycle usage in cities often lead to imbalances among stations over time, resulting in congested stations and others left empty ("hungry stations"), which in turn causes a significant number of dissatisfied customers.

The main issue addressed is that recent methods focus too heavily on spatio-temporal correlations in the data, while overlooking the influence of external conditions (such as weather and time) and the time-varying causal relationships between different regions. Moreover, the presence of incomplete data and random external conditions can result in spurious correlations, making predictions unreliable in certain situations.

To overcome these challenges, STNSCM adopts a causality-based approach. The core idea is to build a causal model to describe traffic prediction, by analyzing the causal relationships among input data, contextual conditions, spatio-temporal states, and prediction outcomes.

The tests that were performed successfully verified the final values obtained by the authors, confirming the excellent performance of the implementation compared with basic approaches on the same dataset.

The reproducibility study also validated the model's interpretability advantages, confirming that the causal graphs generated by STNSCM provide meaningful insights into the relationships between different urban regions and how they are affected by external factors. Additionally, several ablation tests were conducted to quantify the contribution of components and external factors to the model's predictive power.

2 Related works

Spatio-Temporal Traffic Prediction

Early traffic prediction models such as LSTM [3] and GRU [4] focused on temporal patterns but lacked mechanisms to capture spatial dependencies in transportation networks. To address this, models like DCRNN [5] introduced diffusion convolution integrated with GRUs, allowing effective modeling of spatial relations in non-Euclidean graphs. Similarly, STGCN [6] and HGCN [7] used GCNs [8] to learn spatial patterns, with HGCN incorporating a hierarchical structure for multi-scale representation. However, these models relied on static adjacency matrices, limiting their ability to adapt to dynamic traffic changes.

AGCRN [2] improved this by learning adaptive graphs from data, but the graph became fixed after training. GWNet [9] and GMAN [10] introduced attention-based mechanisms to adjust spatial weights dynamically, though still constrained by a predefined graph structure.

Dynamic models like DGCRN [11] and DMSTGCN [12] learned graph structures at each time step, better capturing time-varying dependencies. Yet, they lacked mechanisms to explicitly model external contextual conditions such as weather or holidays, which are critical for real-world performance. Most models simply concatenate such features with the input, failing to account for their confounding effects, which can harm generalization in atypical scenarios.

Causal Learning in Prediction Models

Causal learning offers a principled way to address such confounding by focusing on cause-effect relationships rather than mere correlations. However, traffic prediction often involves unobserved confounders, making backdoor adjustments impractical. The current work addresses this by leveraging the frontdoor criterion, allowing it to remove confounding bias via intermediate representations (spatio-temporal states).

3 Methodology

3.1 Original Paper Method

The overall structure of STNSCM is an encoder-decoder architecture composed of several STNSCU units (Spatio-temporal Neural Structural Causal Unit).

- **Input Gate:** takes as input historical bike flow data (divided by previous week, previous day, and previous four hours) along with corresponding external conditions. The Input Gate aims to extract context-conditioned features, combining historical flow information with relevant external conditions, preparing data for subsequent spatio-temporal analysis.
- **Dynamic Causality Generator:** aims to model the dynamic spatio-temporal causal relationships among different regions, also considering the influence of time-varying external conditions. It takes as input context-conditioned features produced by the Input Gate and the spatio-temporal state from the previous time step. In particular, it assesses the importance of each node (region) and, inspired by self-attention mechanisms, computes similarities between nodes to construct a dynamic causal graph. This graph captures how relationships among regions evolve over time based on bike flow and external conditions.
- **Spatio-temporal Evolutionary Graph Convolution (STEGCN):** is the core component processing spatial and temporal information. It utilizes three graphs to understand the relationships among areas:
 - A graph based on geographical distance between areas. Nearby areas likely influence each other more significantly.
 - A graph based on transition probabilities. Ideally, if many users pick up bikes in one area and drop them in another, there is a strong relationship between these two areas.
 - The dynamic causal graph created by the Dynamic Causality Generator.
- **STNSCU (Spatio-temporal Neural Structural Causal Unit):** integrates the previous three components (Input Gate, Dynamic Causality Generator and STEGCN) to represent the entire process. It uses mechanisms similar to a GRU but replaces simple matrix multiplications with graph convolution operations to update the spatio-temporal state over time. By leveraging historical information, it effectively captures the current state of the bike-sharing system.
- **Counterfactual Representation Reasoning Module:** located between the encoder and decoder, this module extrapolates the state from historical scenarios to future scenarios, considering upcoming external conditions. It takes the state from the encoder and imagines a slightly different future scenario (for example, if tomorrow it rains instead of being sunny). It uses future external conditions to determine how the current state might evolve. Its goal is to enhance the robustness of the model against unexpected changes and improve predictions by considering various future possibilities.

- **Decoder:** receives the initial spatio-temporal state provided by the Counterfactual Representation Reasoning Module and future external conditions to generate bike flow predictions for the subsequent two hours.

3.2 Reproducibility Methodology

We begin the reproducibility challenge by re-running the original STNSCM model as described in the paper. This step allowed us to verify the core implementation and training dynamics. To further assess the effectiveness of the model and understand its contributions compared to simpler or previously proposed architectures, we implemented two other baseline models.

Also, to better understand the role of the structural inputs in the model, we conducted two targeted experiments. In the first one, we removed the geographic affinity matrix and in the second, we removed the OD (origin-destination) matrix. Both matrices have been replaced with an identity matrix to keep self loops while discarding connections between nodes.

Furthermore, we conducted an ablation study to assess the importance of individual components in the STNSCM architecture. By selectively disabling specific modules—such as the dynamic graph generator, the input gate, the Graph-GateRNN and the counterfactual mechanism—we were able to evaluate how each part impacts the model’s overall effectiveness and training dynamics.

Finally, we explored the possibility of applying the model to alternative datasets. Unfortunately, the original codebase lacked the modularity and adaptability required for such extensions, making it impractical to proceed without extensive modifications.

Overall the process it is illustrated in Figure 1.

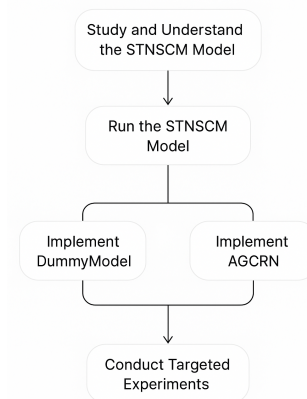


Figure 1. General workflow

4 Implementation

To begin the project, we first resolved several critical issues in the original codebase. These included inconsistent device settings, mismatched naming conventions (e.g., STEGRN instead of STNSCM), and limited documentation. Additionally, many comments were written in Chinese, so we translated all annotations into English and standardized module and variable names. To improve maintainability, we

also reorganized the directory structure by grouping related components and aligning file names with their function.

Once the codebase was cleaned and standardized, we reproduced the original training process on the NYC dataset. Although the original paper presents results for both 30-minute and 60-minute prediction horizons, we focused solely on the 30-minute horizon to reduce execution time and maintain consistency across experiments.

After validating the official model, we implemented two additional models for comparison:

- **DummyModel**, a lightweight GRU-based model that processes temporal sequences from multiple nodes (e.g., bike stations) and generates predictions by repeating the final hidden state. This serves as a minimal benchmark for spatio-temporal forecasting.
- **AGCRN**, an established model cited in the paper, which integrates graph convolution with recurrent networks. It uses learnable node embeddings to construct adaptive graphs and stacked AGCRN cells to encode temporal dependencies, followed by a convolutional decoder.

To make these models compatible, we adapted them to work with the specific data format and preprocessing used in the STNSCM framework.

As mentioned in Section 3, we also performed a series of controlled experiments to evaluate the contribution of specific components of the model. Each experiment corresponds to a particular ablation or modification. The first two are related to modifying the structure of the model itself. To systematically conduct ablation experiments, we introduced a structured approach using a YAML configuration file. Specifically, we defined boolean flags to enable or disable key modules of the model: the Dynamic Graph, Counterfactual Reasoning module, Input Gate, and GraphGateRNN. At runtime, these flags control conditional statements in the codebase that selectively include or exclude certain computations. For example, setting `use_dynamic_graph: false` disables the dynamic causal graph generation, relying solely on static graphs. Similarly, disabling `use_graph_gate_rnn` replaces the gated recurrent update with a simpler static GCN. This approach allowed us to cleanly isolate and quantify the contributions of individual model components.

- **Exp1**, disabled Dynamic Graph and Graph-Gated RNN modules.
- **Exp2**, removed Counterfactual Reasoning and the Input Gate.

While the second two are related to the inputs passed to the model. Specifically, the removed matrices were replaced with an identity matrix to preserve self-loops and neutralize interactions between nodes. This approach allowed us to isolate and evaluate the specific contribution of each matrix to the model’s predictive performance. In particular:

- **Exp3**, replaced the Geo-Affinity matrix with an identity matrix.
- **Exp4**, replaced the OD (Origin-Destination) matrix with an identity matrix.

Figures 2, 3 and Tables 2, 3 use these names to refer to these configurations.

4.1 Datasets

In our work, we encountered challenges when attempting to analyze the dataset used in the paper. The authors only mentioned that they used two datasets named *NYC-Bike* and *BJ-Bike* without providing any direct link or comprehensive description, making it difficult to locate the exact datasets online for reference. We even try to contact the authors to have more details about this but we never received any response.

In the repository there are the datasets but in NPZ format*, which is not directly human-readable. In fact, after extraction, these files are just a collection of tensors, leaving us unable to readily identify key aspects such as attributes and individual rows.

Through careful analysis of the NPZ files, we were able to extract meaningful information about the datasets’ structure and characteristics. As described in the paper, the data appears to be a spatio-temporal traffic flow dataset collected from 64 different monitoring stations. The temporal coverage begins on May 1, 2021 for both dataset as indicated by the timestamp information extracted from the weather data component.

Our examination revealed that the BJ dataset is divided into training (4,362 samples), validation (545 samples), and testing (546 samples) sets. While NYC dataset is divided in the same way but with 13743 items for training and 1718 for validation and test.

Each data point contains historical traffic information spanning 18 time steps with 2 features per station, likely representing inflow and outflow measurements. The target data consists of predictions for 2 future time steps.

Additionally, the datasets incorporates temporal features (6 dimensions) for each time step, which include both time-related (3 dimensions) and weather-related (3 dimensions) information. The datasets also contain positional information for each sample and supplementary weather data, indicating that environmental factors were considered relevant to the traffic prediction task.

4.2 Hyperparameters

For the original paper’s model, we strictly adhered to the hyperparameters reported in the authors’ implementation (learning rate, batch size, hidden dimensions, etc.) to ensure a faithful reproduction of their results. In contrast, the Dummy Model and AGCRN baseline were configured experimentally: we followed a *trial&error* approach on key parameters to find optimal values for our dataset and task. While this approach may not maximize their performance, it ensures a fair comparison by avoiding extensive tuning that could bias results in their favor.

4.3 Experimental setup

We made an initial attempt to run the experiments on our Mac systems (having the computational requirements described in 4.4) and found that the performance was sufficient for our needs, eliminating the need to transfer the code to cloud-based platforms. We did not use Kaggle or Colab since the authors’ code is divided in multiple classes

*NumPy’s compressed archive format

and transporting this into a notebook environment would have required significant time and effort. In addition, we tried to use Lightning AI, but unfortunately we have difficulty running the code under a tight time constraint, so we decided to run it locally on our devices.

All code used for this research, including authors' code and our experiments, is publicly available on our GitHub repository[†]. The repository is structured to facilitate easy replication of our experiments.

4.4 Computational requirements

All experiments were executed on MacBooks Pro equipped with an Apple M2 Pro chip (12-core CPU, 19-core GPU, 16 GB RAM).

Each run (both training and validation) of the STNSCM model on the NYC dataset (200 epochs, 30-minute horizon) took approximately 4 hours to complete, as shown in Table 1. The DummyModel required around 10 minutes, while AGCRN training lasted about 3 hours. Memory usage remained within the 16 GB available, and no external GPU was required.

Table 1. Resources Usage STNSCM model

Metric	Value
Device used	mps
Total training time	3.55 hours
Total validation time	0.13 hours
Peak GPU memory usage	2.05 GB
Peak CPU memory usage	0.52 GB
Total model parameters	419218

5 Results

The authors present the results of the original paper using three evaluation metrics: MAE (Mean Absolute Error), RMSE (Root Mean Squared Error) and MAPE (Mean Absolute Percentage Error). Our results are summarized in Table 2 and Figure 2:

Table 2. Experimental results

Methods	MAE	RMSE	MAPE
STNSCM	2.5289	4.6835	19.8475%
AGCRN	10.6209	20.3866	67.041%
Dummy	3.7620	7.7303	29.8141%

As shown in Table 2, the STNSCM model significantly outperforms both the AGCRN baseline and our simple Dummy model across all evaluation metrics by achieving the lowest MAE, RMSE, and MAPE values. These results confirm the strong predictive capability and robustness claimed by the authors. The superior performance highlights the advantages of integrating causal reasoning and dynamic spatio-temporal modeling in bike flow prediction. For fairness, it should be noted that the relatively poor results obtained by AGCRN can be attributed to the fact that this model was not originally designed for this dataset; we had to modify the model to adapt it to the dataset under study.

Figure 2. Losses of Models

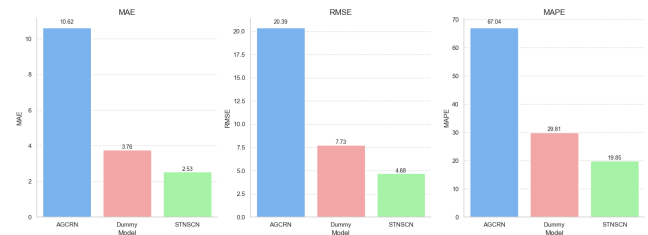
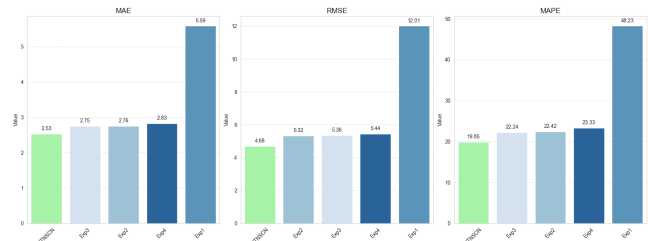


Table 3. Ablation tests results (average of 3 runs)

Methods	MAE	RMSE	MAPE
STNSCM	2.5289	4.6835	19.8475%
Exp1	5.5859	12.0095	48.2285%
Exp2	2.7573	5.3226	22.4174%
Exp3	2.7516	5.3599	22.2372%
Exp4	2.8307	5.4357	23.3339%

Figure 3. Losses on Ablation Tests



The results shown in Table 3 and 3 highlight the critical role of the Dynamic Graph and Graph-Gated RNN modules.

- When these components are disabled (Exp1), performance drops drastically across all metrics, with MAE, RMSE, and MAPE almost doubling compared to the full model. This evidence that modeling dynamic spatio-temporal causality is essential for accurate bike flow prediction.
- Disabling the Counterfactual Reasoning and Input Gate (Exp2) also leads to a modest degradation in performance, in fact the impact is less severe than in Exp1. This indicates that incorporating future contextual conditions and properly conditioning the input features are important for robustness and stability, particularly under varying external scenarios.
- Removing either the Geographic Matrix (Exp3) or the OD Transition Matrix (Exp4) results in a moderate but consistent decrease in performance. This shows that while static spatial features contribute to the overall prediction quality, they are somewhat less critical compared to the dynamic and counterfactual components.

Overall, these ablation tests confirm that each component of STNSCM plays a meaningful role, with the dynamic

[†]<https://github.com/cricci3/STNSCM>

graph modeling and counterfactual reasoning being the most crucial for achieving state-of-the-art performance.

6 Discussion and conclusion

In this project, we successfully reproduced the results of the STNSCM model and verified its strong performance in spatio-temporal bike flow prediction. Our replication confirmed the effectiveness of incorporating causal reasoning and dynamic graph structures, as originally proposed. The model consistently outperformed both a naive GRU-based DummyModel and the more advanced AGCRN baseline across all standard evaluation metrics.

Through ablation studies, we systematically evaluated the contribution of each component within the STNSCM architecture. Results demonstrated that the dynamic graph and Graph-Gated RNN modules are essential for modeling spatio-temporal causality, while the counterfactual reasoning mechanism and structural matrices (Geo and OD) also provide significant improvements in prediction accuracy and robustness.

Despite the overall success, we encountered several challenges during implementation. The original codebase was poorly documented and tightly coupled, making it difficult to adapt the model to new datasets or perform modular experiments. Additionally, the lack of public dataset descriptions limited our ability to fully interpret and generalize the results.

Future work could focus on refactoring the model into a more modular and extensible framework to support broader experimentation. It would also be valuable to evaluate the model on additional cities or transportation systems, assess its performance under severe distribution shifts, and explore alternative causal modeling strategies. Overall, our findings validate the original claims and reinforce the importance of integrating causal reasoning in spatio-temporal forecasting models.

References

- [1] Pan Deng, Yu Zhao, Juntong Liu, Xiaofeng Jia, and Mulan Wang. Spatio-temporal neural structural causal models for bike flow prediction, 2023. URL <https://arxiv.org/abs/2301.07843>.
- [2] Lei Bai, Lina Yao, Can Li, Xianzhi Wang, and Can Wang. Adaptive graph convolutional recurrent network for traffic forecasting, 2020. URL <https://arxiv.org/abs/2007.02842>.
- [3] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9:1735–1780, 11 1997. doi: 10.1162/neco.1997.9.8.1735.
- [4] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation, 2014. URL <https://arxiv.org/abs/1406.1078>.
- [5] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting, 2018. URL <https://arxiv.org/abs/1707.01926>.
- [6] Bing Yu, Haoteng Yin, and Zhanxing Zhu. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence, IJCAI-2018*, page 3634–3640. International Joint Conferences on Artificial Intelligence Organization, July 2018. doi: 10.24963/ijcai.2018/505. URL <http://dx.doi.org/10.24963/ijcai.2018/505>.
- [7] Kan Guo, Yongli Hu, Yanfeng Sun, Sean Qian, Junbin Gao, and Baocai Yin. Hierarchical graph convolution network for traffic forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35:151–159, 05 2021. doi: 10.1609/aaai.v35i1.16088.
- [8] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks, 2017. URL <https://arxiv.org/abs/1609.02907>.
- [9] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. Graph wavenet for deep spatial-temporal graph modeling, 2019. URL <https://arxiv.org/abs/1906.00121>.
- [10] Chuanpan Zheng, Xiaoliang Fan, Cheng Wang, and Jianzhong Qi. Gman: A graph multi-attention network for traffic prediction, 2019. URL <https://arxiv.org/abs/1911.08415>.
- [11] Fuxian Li, Jie Feng, Huan Yan, Guangyin Jin, Depeng Jin, and Yong Li. Dynamic graph convolutional recurrent network for traffic prediction: Benchmark and solution, 2021. URL <https://arxiv.org/abs/2104.14917>.
- [12] Liangzhe Han, Bowen Du, Leilei Sun, Yanjie Fu, Yisheng Lv, and Hui Xiong. Dynamic and multi-faceted spatio-temporal deep learning for traffic speed forecasting. pages 547–555, 08 2021. doi: 10.1145/3447548.3467275.